Python & NumPy Handbook

Complete Reference for Machine Intelligence Course

Andrés Masegosa - Fall 2025

Quick Navigation: Click section number to jump, use PDF back button to return

Contents

1 Python Language Fundamentals **3**

1.1 Variables and Assignment . . . 3

1.2 Functions . . . 4

1.3 Loops and Iterations . . . 5

1.4 String Formatting . . . 6

1.5 Comparison and Boolean Operations . . . 7

1.6 Conditional Statements . . . 8

2 NumPy Arrays - Creation and Basic Operations **9**

2.1 Creating Arrays . . . 9

2.2 Array Attributes . . . 11

3 NumPy Array Indexing and Slicing **12**

3.1 Basic Indexing . . . 12

3.2 Boolean Indexing . . . 14

3.3 Advanced Indexing for Plotting . . . 15

4 NumPy Mathematical Operations **16**

4.1 Element-wise Operations . . . 16

4.2 Matrix Operations . . . 17

4.3 Aggregation Functions . . . 18

4.4 Mathematical Functions . . . 20

5 Random Number Generation **23**

6 Loading and Processing Data **26**

7 Machine Learning Specific Operations **29**

7.1 Vectorization for Predictions . . . 29

7.2 Gradient Computation 31
7.3 Cost Functions 32
7.4 Regularization 34

8 Deep Copy **35**

9 Visualization with Matplotlib **36**
9.1 Basic Plotting 36

10 Common Course Patterns **39**
10.1 Complete Training Loop 39
10.2 Prediction and Accuracy 40

11 Debugging and Best Practices **41**

12 Quick Reference Tables **41**
12.1 NumPy Functions Summary 41
12.2 Common Shapes 42
12.3 Axis Parameter 42

13 Appendix: Broadcasting Rules **43**

14 Summary **44**

1 Python Language Fundamentals

1.1 Variables and Assignment

Variable Assignment

Basic Assignment:

```
x = 5            # Integer
y = 3.14         # Float
name = "Alice"   # String
data = [1,2,3]   # List
```

Multiple Assignment:

```
a, b = 10, 20
X_train, y_train = load_data()  # Unpack function return values
```

When Used in Course:

- Loading data: `X, y = load_data_1()`
- Gradient computation: `dj_db, dj_dw = compute_gradient(X, y, w, b)`
- Splitting data: `X_train, y_train, X_val, y_val = split_data(X, y)`

Plain English: You can assign multiple variables at once. Very useful when functions return multiple values (like gradients for both `w` and `b`).

1.2 Functions

Function Definition

Basic Syntax:

```python
def function_name(param1, param2, param3=default_value):
    """
    Docstring: Describe what function does

    Args:
        param1 (type): Description
        param2 (type): Description
        param3 (type): Description (optional)

    Returns:
        return_type: Description
    """
    # Function body
    result = param1 + param2
    return result
```

Course Example:

```python
def sigmoid(z):
    """
    Compute the sigmoid of z

    Args:
        z (ndarray): A scalar, numpy array of any size.

    Returns:
        g (ndarray): sigmoid(z), with the same shape as z
    """
    g = 1/(1 + np.exp(-z))
    return g
```

Default Parameters:

```python
def compute_cost(X, y, w, b, lambda_=1):
    # If lambda_ not provided, uses 1 as default
    ...
```

Plain English: Functions are reusable blocks of code. The docstring explains what it does. Parameters with `=` have default values. The function returns values using `return`.

1.3 Loops and Iterations

For Loops

Iterate Over Range:

```
for i in range(5):          # i = 0, 1, 2, 3, 4
    print(i)

for i in range(1, 10):      # i = 1, 2, ..., 9
    print(i)

for i in range(0, 10, 2):   # i = 0, 2, 4, 6, 8 (step=2)
    print(i)
```

Course Usage - Gradient Descent:

```
for i in range(num_iters):
    # Calculate gradient and update parameters
    dj_db, dj_dw = gradient_function(X, y, w, b, lambda_)
    w = w - alpha * dj_dw
    b = b - alpha * dj_db
```

Course Usage - Grid Search:

```
for i in range(len(u)):
    for j in range(len(v)):
        z[i,j] = sig(np.dot(map_feature(u[i], v[j]), w) + b)
```

Plain English: `for i in range(n)` loops from 0 to n-1. `range(start, stop)` loops from start to stop-1. In the course, loops are used for iterations in gradient descent and nested loops for grid computations.

1.4 String Formatting

F-Strings (Formatted String Literals)

Basic Usage:

```
name = "Alice"
age = 25
print(f"My name is {name} and I am {age} years old")
# Output: My name is Alice and I am 25 years old
```

Number Formatting:

```
cost = 0.693147
print(f"Cost: {cost:.3f}")       # 3 decimal places: 0.693
print(f"Cost: {cost:.2f}")       # 2 decimal places: 0.69
print(f"Iteration {i:4d}")       # 4-digit integer, padded: "   5"
print(f"Accuracy: {acc:8.2f}%")  # 8 chars wide, 2 decimals
```

Course Example:

```
print(f"Iteration {i:4d}: Training Cost {J_train[-1]:8.2f}, "
      f"Train Accuracy {acc_train[-1]:8.2f}, "
      f"Test Accuracy {acc_test[-1]:8.2f}")
# Output: Iteration   10: Training Cost     0.52, Train Accuracy
   85.00, Test Accuracy    83.50
```

Format Specifiers:

- `:.2f` - floating point, 2 decimal places
- `:4d` - integer, 4 characters wide
- `:8.2f` - float, 8 chars wide, 2 decimals

Plain English: F-strings let you embed variables directly in strings using {`variable`}. Use `:.nf` to control decimal places, `:wd` for integer width. This makes output clean and aligned.

1.5 Comparison and Boolean Operations

Comparison Operators

Comparisons:

```
x == y     # Equal to
x != y     # Not equal to
x > y      # Greater than
x < y      # Less than
x >= y     # Greater than or equal to
x <= y     # Less than or equal to
```

Course Usage - Boolean Indexing:

```
positive = (y_train == 1)  # Boolean array: True where y==1
negative = (y_train == 0)  # Boolean array: True where y==0

# Use for plotting different classes
plt.plot(X[positive, 0], X[positive, 1], 'k+', label="Admitted")
plt.plot(X[negative, 0], X[negative, 1], 'yo', label="Not admitted")
```

Course Usage - Predictions:

```
p = (sigmoid(np.dot(X, w) + b) >= 0.5)  # True if >=0.5, else False
accuracy = np.mean(p == y) * 100        # Percentage of correct
    predictions
```

Plain English: Comparisons return `True` or `False`. When applied to NumPy arrays, you get boolean arrays. These can be used to filter/select data or count correct predictions.

1.6 Conditional Statements

If-Elif-Else

Basic Syntax:

```
if condition:
    # Do this if condition is True
elif another_condition:
    # Do this if first is False but this is True
else:
    # Do this if all above are False
```

Course Example:

```
if i < 100000:      # prevent resource exhaustion
    J_history_train.append(cost_function(X_train, y_train, w, b))
    J_history_train_acc.append(accuracy(X_train, y_train, w, b))

if i % math.ceil(num_iters / 50) == 0:  # Print every 1/50 iterations
    print(f"Iteration {i:4d}: Training Cost {J_history_train[-1]:8.2f}"
        )
```

Ternary Operator:

```
result = "Yes" if finished else "No"
# Same as:
# if finished:
#     result = "Yes"
# else:
#     result = "No"
```

Plain English: `if` checks a condition. If True, runs that block. `elif` (else-if) checks another condition if first was False. `else` runs if nothing matched. The `%` operator (modulo) checks divisibility.

2 NumPy Arrays - Creation and Basic Operations

2.1 Creating Arrays

np.array() - Create Array from List

Syntax:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
```

Course Examples:

```
# 1D array (vector)
w = np.array([1, 10])
x = np.array([[1, 0]])  # 2D array with 1 row, 2 columns

# 2D array (matrix)
X = np.array([[1, 2], [3, 4], [5, 6]])
```

What You Get:

```
arr.shape     # (5,) - 1D array with 5 elements
X.shape       # (3, 2) - 2D array with 3 rows, 2 columns
```

Plain English: `np.array()` converts a Python list into a NumPy array. Use nested lists `[[ ]]` for 2D arrays (matrices). The shape tells you dimensions: `(m, n)` means m rows, n columns.

np.zeros() - Create Array of Zeros

Syntax:

```
zeros_1d = np.zeros(5)          # 1D array: [0. 0. 0. 0. 0.]
zeros_2d = np.zeros((3, 4))     # 2D: 3 rows, 4 columns, all zeros
```

Course Usage:

```
dj_dw = np.zeros(w.shape)  # Initialize gradient to same shape as w
z = np.zeros((len(u), len(v)))  # Grid for decision boundary
```

Key Point: For 2D, use tuple: `np.zeros((rows, cols))` - notice the double parentheses!
Plain English: Creates array filled with zeros. Used to initialize gradients, accumulators, or grids before filling them with computed values.

np.arange() - Create Sequence

Syntax:

```
np.arange(stop)              # 0, 1, 2, ..., stop-1
np.arange(start, stop)       # start, start+1, ..., stop-1
np.arange(start, stop, step) # start, start+step, ..., < stop
```

Course Examples:

```
x = np.arange(-10, 10, 0.1)  # From -10 to 10, step 0.1
# Result: [-10. , -9.9, -9.8, ..., 9.8, 9.9]

it = np.arange(0, num_iters, 1)  # Iteration numbers for plotting
plt.plot(it, J_history_train, label="Training Cost")
```

Plain English: Like Python's `range()`, but returns NumPy array and works with floats. Used for creating x-axis values for plots or sequences of numbers.

np.linspace() - Linearly Spaced Values

Syntax:

```
np.linspace(start, stop, num)  # num evenly spaced values from start to
    stop
```

Course Example:

```
u = np.linspace(-2.5, 2.5, 50)  # 50 points from -2.5 to 2.5
v = np.linspace(-2.5, 2.5, 50)
# Used to create grid for decision boundary visualization
```

Difference from arange:

- `arange`: specify step size, number of points varies
- `linspace`: specify number of points, step size varies

Plain English: Creates evenly spaced numbers. You specify how many points you want, not the step size. Perfect for creating smooth grids for plotting.

2.2 Array Attributes

.shape - Get/Check Array Dimensions

Syntax:

```
arr.shape  # Returns tuple of dimensions
```

Course Examples:

```
X_train.shape      # (100, 2) - 100 examples, 2 features
y_train.shape      # (100,) - 100 labels (1D array)
w.shape            # (2,) - 2 weights

m, n = X.shape     # m = number of examples, n = number of features
```

Common Patterns:

```
# Check if array is empty
if X.shape[0] == 0:
    print("No data")

# Get number of examples
m = X_train.shape[0]

# Get number of features
n = X_train.shape[1]
```

Plain English: `.shape` returns dimensions as a tuple. For 2D: `(rows, cols)`. For 1D: `(length,)`. The comma in `(100,)` indicates 1D array (not 2D with 1 column).

.reshape() - Change Array Shape

Syntax:

```
arr.reshape(new_shape)
arr.reshape(-1, 1)  # -1 means "infer this dimension"
```

Course Example:

```
initial_w = 0.01 * (np.random.rand(2).reshape(-1, 1) - 0.5)
# Creates (2,) array, reshapes to (2, 1) column vector
```

Common Reshapes:

```
x = np.array([1, 2, 3, 4, 5, 6])  # shape: (6,)

x.reshape(2, 3)    # [[1,2,3], [4,5,6]], shape: (2,3)
x.reshape(3, 2)    # [[1,2], [3,4], [5,6]], shape: (3,2)
x.reshape(-1, 1)   # [[1],[2],[3],[4],[5],[6]], shape: (6,1) column
x.reshape(1, -1)   # [[1,2,3,4,5,6]], shape: (1,6) row
```

Plain English: Changes shape without changing data. Total elements must stay same. Use `-1` to let NumPy figure out that dimension. Useful for converting vectors to matrices.

3 NumPy Array Indexing and Slicing

3.1 Basic Indexing

Accessing Elements

1D Arrays:

```
arr = np.array([10, 20, 30, 40, 50])

arr[0]      # First element: 10
arr[1]      # Second element: 20
arr[-1]     # Last element: 50
arr[-2]     # Second to last: 40
```

2D Arrays:

```
X = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

X[0, 0]     # Row 0, Column 0: 1
X[0, 2]     # Row 0, Column 2: 3
X[1, 1]     # Row 1, Column 1: 5
X[-1, -1]   # Last row, last column: 9
```

Course Example:

```
print("First five elements in X_train are:\n", X_train[:5])
print(f"x^(1) = {X_train[0]}, y^(1) = {y_train[0]}")
```

Plain English: Use `[i]` for 1D arrays, `[i, j]` for 2D. Indices start at 0. Negative indices count from end.

Slicing Arrays

Syntax: `arr[start:stop:step]`

- `start`: beginning index (inclusive)
- `stop`: ending index (exclusive)
- `step`: step size (default 1)

1D Slicing:

```
arr = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

arr[2:5]       # [2, 3, 4] - indices 2, 3, 4
arr[:5]        # [0, 1, 2, 3, 4] - first 5 elements
arr[5:]        # [5, 6, 7, 8, 9] - from index 5 to end
arr[::2]       # [0, 2, 4, 6, 8] - every 2nd element
arr[::-1]      # [9, 8, 7, ..., 0] - reversed!
```

2D Slicing:

```
X = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

X[0:2, :]      # First 2 rows, all columns
               # [[1,2,3], [4,5,6]]

X[:, 0:2]      # All rows, first 2 columns
               # [[1,2], [4,5], [7,8]]

X[1:, 1:]      # Rows from 1 onwards, columns from 1 onwards
               # [[5,6], [8,9]]
```

Course Examples:

```
# Load features and labels from data
X = data[:, :2]    # All rows, first 2 columns (features)
y = data[:, 2]     # All rows, 3rd column (labels)

# Split train/test
X_split_train = X[0:size_train, :]       # First size_train rows
X_split_val = X[size_train:, :]          # Remaining rows

# Print first 5 examples
print(X_train[:5])
print(y_train[:5])
```

Plain English: `start:stop` gets elements from start up to (but not including) stop. Omitting start means from beginning. Omitting stop means to end. Use `:` alone to mean "all".

3.2 Boolean Indexing

Boolean Masks

Concept: Use boolean array to select elements
Course Examples:

```
# Create boolean masks
positive = (y_train == 1)  # True where y==1, False otherwise
negative = (y_train == 0)  # True where y==0, False otherwise

# Use masks to filter data
X_positive = X_train[positive]  # Only rows where y==1
X_negative = X_train[negative]  # Only rows where y==0

# Plot different classes
plt.plot(X_train[positive, 0], X_train[positive, 1], 'k+')
plt.plot(X_train[negative, 0], X_train[negative, 1], 'yo')
```

How It Works:

```
arr = np.array([10, 20, 30, 40, 50])
mask = arr > 25                # [False, False, True, True, True]
result = arr[mask]             # [30, 40, 50]
```

Course Pattern:

```
# Compute predictions
predictions = (sigmoid(np.dot(X, w) + b) >= 0.5)  # Boolean array

# Compute accuracy
accuracy = np.mean(predictions == y) * 100
# predictions == y creates boolean array showing correct predictions
# np.mean() treats True=1, False=0, giving fraction correct
```

Plain English: Boolean indexing selects elements where condition is True. Create mask with comparison (==, >, etc.), then use mask as index. Very powerful for filtering data by class or condition.

3.3 Advanced Indexing for Plotting

Selecting Columns for Plotting

Course Pattern:

```
# X has shape (m, n) - m examples, n features
# Select specific columns for x and y axes

plt.plot(X[:, 0], X[:, 1], 'o')  # Column 0 vs Column 1

# With boolean mask
positive = y == 1
plt.plot(X[positive, 0], X[positive, 1], 'k+', label="Class 1")
plt.plot(X[negative, 0], X[negative, 1], 'yo', label="Class 0")
```

Breaking Down: `X[positive, 0]`

1. `positive` is boolean array of shape `(m,)`
2. `X[positive, :]` selects rows where positive==True
3. `X[positive, 0]` further selects only column 0 of those rows

Plain English: First filter by rows (using boolean mask), then select column. This gets you specific feature values for specific class. Essential for plotting different classes with different markers/colors.

4 NumPy Mathematical Operations

4.1 Element-wise Operations

Basic Arithmetic

Operations on Arrays:

```
a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])

a + b      # [11, 22, 33, 44] - element-wise addition
a - b      # [-9, -18, -27, -36] - element-wise subtraction
a * b      # [10, 40, 90, 160] - element-wise multiplication
a / b      # [0.1, 0.1, 0.1, 0.1] - element-wise division
a ** 2     # [1, 4, 9, 16] - element-wise power
```

Operations with Scalars:

```
a = np.array([1, 2, 3, 4])

a + 10     # [11, 12, 13, 14] - add 10 to each element
a * 2      # [2, 4, 6, 8] - multiply each by 2
a - 5      # [-4, -3, -2, -1] - subtract 5 from each
```

Course Examples:

```
# Z-score normalization
X_norm = (X - X_mean) / X_sigma

# Add noise to data
X = X + np.random.normal(0, 0.25, size=X.shape)

# Scale weights
initial_w = 0.01 * (np.random.rand(2) - 0.5)

# Compute predictions
f_wb = w * x + b  # Element-wise: multiply w*x, add b to all
```

Plain English: Operators `+, -, *, /` work element-by-element on arrays. When combining array with scalar, scalar is applied to every element (called broadcasting).

4.2 Matrix Operations

np.dot() - Dot Product / Matrix Multiplication

Syntax:

```
result = np.dot(a, b)
```

Case 1: Vector Dot Product (both 1D):

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
result = np.dot(a, b)  # 1*4 + 2*5 + 3*6 = 32
```

Case 2: Matrix-Vector Multiplication:

```
X = np.array([[1, 2],
              [3, 4],
              [5, 6]])   # shape: (3, 2)
w = np.array([0.5, 0.3])  # shape: (2,)

result = np.dot(X, w)
# result[0] = 1*0.5 + 2*0.3 = 1.1
# result[1] = 3*0.5 + 4*0.3 = 2.7
# result[2] = 5*0.5 + 6*0.3 = 4.3
# result = [1.1, 2.7, 4.3], shape: (3,)
```

Course Usage - Model Predictions:

```
# Compute f(x) = w x + b for all examples
f_wb = np.dot(X, w) + b

# X shape: (m, n) - m examples, n features
# w shape: (n,) - n weights
# Result shape: (m,) - m predictions
```

Course Usage - Gradient Computation:

```
# Gradient for w
err = f_w - y                  # shape: (m,)
dj_dw = (1/m) * np.dot(X.T, err)
# X.T shape: (n, m)
# err shape: (m,)
# Result shape: (n,) - gradient for each weight
```

Course Usage - Logistic Regression:

```
z = np.dot(X, w) + b           # Linear combination
f_w = sigmoid(z)                # Apply sigmoid
```

Plain English: `np.dot()` performs mathematical dot product. For vectors: sum of element-wise products. For matrix-vector: each row of matrix dotted with vector. This is THE core operation for computing predictions and gradients in machine learning!

4.3 Aggregation Functions

np.sum() - Sum Elements

Syntax:

```
np.sum(arr)                # Sum all elements
np.sum(arr, axis=0)        # Sum along axis 0 (column-wise)
np.sum(arr, axis=1)        # Sum along axis 1 (row-wise)
```

Examples:

```
arr = np.array([[1, 2, 3],
                [4, 5, 6]])

np.sum(arr)            # 21 (all elements)
np.sum(arr, axis=0)    # [5, 7, 9] (sum each column)
np.sum(arr, axis=1)    # [6, 15] (sum each row)
```

Course Usage:

```
# Sum squared weights (regularization)
reg_cost = (lambda_/(2*m)) * np.sum(np.square(w))

# Sum errors for gradient (bias term)
dj_db = (1/m) * np.sum(err)

# Count correct predictions (True=1, False=0)
num_correct = np.sum(predictions == y)
```

Plain English: `np.sum()` adds up elements. No axis: sum everything. `axis=0`: sum down columns (result has shape of 1 row). `axis=1`: sum across rows (result has shape of 1 column).

np.mean() - Compute Average

Syntax:

```
np.mean(arr)               # Mean of all elements
np.mean(arr, axis=0)       # Mean of each column
np.mean(arr, axis=1)       # Mean of each row
```

Course Usage - Normalization:

```
X_mean = np.mean(X, axis=0)  # Mean of each feature (column)
# X shape: (m, n)
# X_mean shape: (n,) - one mean per feature
```

Course Usage - Cost Function:

```
total_cost = -np.mean(y * np.log(f_w) + (1-y) * np.log(1-f_w))
# Computes average over all examples
```

Course Usage - Accuracy:

```
accuracy = np.mean(p == y) * 100
# p == y is boolean array (True/False)
# np.mean treats True=1, False=0
# Result is fraction correct, multiply by 100 for percentage
```

Plain English: `np.mean()` computes average. For boolean arrays, it gives fraction of True values (since True=1, False=0). Perfect for computing accuracy!

np.std() - Standard Deviation

Syntax:

```
np.std(arr)                # Standard deviation of all elements
np.std(arr, axis=0)        # Std dev of each column
```

Course Usage - Normalization:

```
def zscore_normalize_features(X):
    X_mean = np.mean(X, axis=0)      # Mean of each feature
    X_sigma = np.std(X, axis=0)      # Std dev of each feature
    X_norm = (X - X_mean) / X_sigma  # Z-score normalization
    return X_norm
```

Why Normalize:

- Features on different scales can slow gradient descent
- Normalization makes all features have mean=0, std=1
- Helps gradient descent converge faster

Plain English: Standard deviation measures spread of data. Used in z-score normalization to standardize features. After normalization, all features have comparable scales.

4.4 Mathematical Functions

np.exp() - Exponential Function

Syntax:

```
np.exp(x)  # Computes e^x element-wise
```

Examples:

```
np.exp(0)       # 1.0
np.exp(1)       # 2.718... (e)
np.exp(2)       # 7.389...
np.exp(-1)      # 0.368...

arr = np.array([0, 1, 2])
np.exp(arr)     # [1.0, 2.718, 7.389]
```

Course Usage - Sigmoid Function:

```
def sigmoid(z):
    g = 1 / (1 + np.exp(-z))
    return g

# If z is array, np.exp(-z) computed element-wise
z = np.array([-1, 0, 1])
sigmoid(z)  # [0.269, 0.5, 0.731]
```

Why Important:

- Sigmoid function uses exp: $\sigma(z) = \frac{1}{1+e^{-z}}$
- Core of logistic regression
- Converts any number to (0, 1) range

Plain English: `np.exp(x)` computes e^x where $e \approx 2.718$. Applied element-wise to arrays. Essential for sigmoid function in logistic regression.

np.log() - Natural Logarithm

Syntax:

```
np.log(x)  # Computes ln(x) element-wise
```

Examples:

```
np.log(1)       # 0.0
np.log(np.e)    # 1.0 (ln(e) = 1)
np.log(10)      # 2.302...

arr = np.array([1, 2, 10])
np.log(arr)     # [0.0, 0.693, 2.302]
```

Course Usage - Logistic Regression Cost:

```
# Cost function for logistic regression
total_cost = -np.mean(y * np.log(f_w) + (1-y) * np.log(1-f_w))

# y and f_w are arrays
# np.log applied element-wise
# When y=1: cost includes -log(f_w)
# When y=0: cost includes -log(1-f_w)
```

Warning:

```
# np.log(0) gives -inf (negative infinity)
# np.log(negative) gives nan (not a number)
# To avoid: clip predictions away from 0 and 1
```

Plain English: `np.log(x)` is natural logarithm (base e). Used in logistic regression cost function. The $-\log$ term heavily penalizes wrong confident predictions.

np.square() - Square Elements

Syntax:

```
np.square(arr)  # Element-wise: arr**2
```

Course Usage - Regularization:

```
# L2 regularization term
reg_cost = (lambda_/(2*m)) * np.sum(np.square(w))

# w is weight vector
# np.square(w) squares each weight
# np.sum adds them up
# This penalizes large weights
```

Alternative:

```
np.square(arr)  # Same as
arr ** 2         # Same as
arr * arr
```

Plain English: Squares each element. Used in regularization to penalize large weights. Same as `**2` but sometimes clearer.

np.clip() - Limit Values

Syntax:

```
np.clip(arr, min_val, max_val)
```

Examples:

```
arr = np.array([-5, 0, 5, 10, 15])
np.clip(arr, 0, 10)  # [0, 0, 5, 10, 10]
# Values < 0 become 0
# Values > 10 become 10
```

Course Usage - Numerical Stability:

```
# Prevent overflow/underflow in exponential
logit = np.clip(np.dot(X, w) + b, -10, 10)
f_w = sigmoid(logit)

# Without clipping:
# Very large positive z -> exp(-z) underflows to 0
# Very large negative z -> exp(-z) overflows
```

Why Needed:

- `exp(100)` = overflow (too large)
- `exp(-100)` = underflow (too close to 0)
- Clipping to [-10, 10] keeps values manageable
- Sigmoid still works correctly in this range

Plain English: `np.clip()` limits values to a range. Values below min become min, values above max become max. Used to prevent numerical errors in exponentials and logarithms.

5 Random Number Generation

np.random.seed() - Set Random Seed

Syntax:

```
np.random.seed(seed_value)
```

Course Usage:

```
np.random.seed(1)
X = X + np.random.normal(0, 0.25, size=X.shape)
# Adding noise is reproducible with same seed

np.random.seed(123)
perm = np.random.permutation(X.shape[0])
# Shuffling is reproducible
```

Why Important:

- Makes random operations reproducible
- Same seed -¿ same "random" numbers
- Essential for debugging and comparing results
- Teacher uses this so everyone gets same results

Plain English: Sets the starting point for random number generation. Same seed always produces same sequence of "random" numbers. Use it to make experiments reproducible!

np.random.rand() - Uniform Random [0

Syntax:

```
np.random.rand(d0, d1, ..., dn)  # Returns array of given shape
```

Examples:

```
np.random.rand()         # Single random float in [0, 1)
np.random.rand(5)        # Array of 5 random values
np.random.rand(2, 3)     # 2x3 array of random values
```

Course Usage - Initialize Weights:

```
# Random initialization near zero
np.random.seed(10)
initial_w = np.random.rand(X_train.shape[1])

# Random initialization with scaling
initial_w = 0.01 * (np.random.rand(2).reshape(-1,1) - 0.5)
# Creates values in range [-0.005, 0.005]
```

Plain English: Generates random numbers uniformly distributed between 0 and 1. Used to initialize weights randomly. Multiplying/subtracting changes the range.

np.random.normal() - Gaussian/Normal Distribution

Syntax:

```
np.random.normal(mean, std, size=shape)
```

Examples:

```
# Single value, mean=0, std=1
np.random.normal(0, 1)

# 100 values, mean=5, std=2
np.random.normal(5, 2, size=100)

# 2D array
np.random.normal(0, 1, size=(3, 4))
```

Course Usage - Add Noise:

```
# Add Gaussian noise to features
np.random.seed(1)
X = X + np.random.normal(0, 0.25, size=X.shape)

# Mean = 0 (no shift)
# Std = 0.25 (amount of noise)
# size=X.shape makes noise array same shape as X
```

Plain English: Generates random numbers from normal (bell curve) distribution. Specify mean (center) and standard deviation (spread). Used to add realistic noise to data.

np.random.permutation() - Shuffle Indices

Syntax:

```
perm = np.random.permutation(n)  # Random permutation of 0 to n-1
```

Examples:

```
np.random.permutation(5)  # e.g., [3, 1, 4, 0, 2]
np.random.permutation(10) # e.g., [7, 2, 8, 1, 9, 0, 5, 4, 3, 6]
```

Course Usage - Shuffle Data:

```
def split_data(X, y):
    # Permutate the data
    np.random.seed(123)
    perm = np.random.permutation(X.shape[0])  # Random order of indices
    X = X[perm, :]  # Reorder rows
    y = y[perm]     # Reorder labels same way

    # Split into train/test
    size_train = int(0.66 * X.shape[0])
    X_train = X[0:size_train, :]
    X_test = X[size_train:, :]
    ...
```

Why Shuffle:

- Data might be ordered by class
- Shuffling ensures random train/test split
- Prevents bias in splitting

Plain English: Creates random ordering of numbers 0 to n-1. Use these as indices to shuffle rows of arrays. Critical for creating randomized train/test splits.

6 Loading and Processing Data

np.loadtxt() - Load Data from Files

Syntax:

```
data = np.loadtxt(filename, delimiter=',', skiprows=0)
```

Parameters:

- `filename`: path or URL to file
- `delimiter`: character separating values (usually ',')
- `skiprows`: number of header rows to skip

Course Example:

```
def load_data_1():
    # Load from URL
    url = "https://drive.google.com/uc?id=1
        AerETIM3wPHSM8oP3MyIT5q5l6rZd0Ei"
    data = np.loadtxt(url, delimiter=',', skiprows=0)

    # Split into features and labels
    X = data[:, :2]    # First 2 columns
    y = data[:, 2]     # Third column

    # Add noise
    np.random.seed(1)
    X = X + np.random.normal(0, 0.25, size=X.shape)

    # Normalize
    X = zscore_normalize_features(X)

    return X, y
```

Plain English: Loads data from CSV file into NumPy array. Each row becomes a row in array. Use slicing to separate features (X) from labels (y). Works with files or URLs!

Z-Score Normalization Pattern

Complete Implementation:

```python
def zscore_normalize_features(X):
    """
    Normalizes features to have mean=0 and std=1

    Args:
      X (ndarray (m,n)): input data, m examples, n features

    Returns:
      X_norm (ndarray (m,n)): normalized data
    """
    # Mean of each column (feature)
    X_mean = np.mean(X, axis=0)      # Mean of each feature

    # Standard deviation of each column
    X_sigma = np.std(X, axis=0)      # Std dev of each feature

    # Normalize: subtract mean, divide by std
    X_norm = (X - X_mean) / X_sigma  # Z-score normalization

    return X_norm                     # Return normalized data
```

Why Normalize:

- Different features may have different scales
- Example: size (100-5000) vs bedrooms (1-10)
- Large scale features dominate gradient descent
- Normalization makes all features comparable
- Gradient descent converges much faster

Broadcasting Explanation:

```python
# X shape: (100, 2) - 100 examples, 2 features
# X_mean shape: (2,) - mean of each feature
# X - X_mean: NumPy broadcasts X_mean to (100, 2)
#             subtracts same mean from each row
```

Plain English: Z-score normalization standardizes features to mean=0, std=1. Compute mean and std of each column, then subtract mean and divide by std. Broadcasting handles dimension mismatch automatically!

Train/Test Split Pattern

Complete Implementation:

```python
def split_data(X, y):
    # 1. Shuffle data
    np.random.seed(123)
    perm = np.random.permutation(X.shape[0])
    X = X[perm, :]
    y = y[perm]

    # 2. Compute split point (66% train, 34% test)
    size_train = int(0.66 * X.shape[0])

    # 3. Split
    X_split_train = X[0:size_train, :]
    y_split_train = y[0:size_train]

    X_split_val = X[size_train:, :]
    y_split_val = y[size_train:]

    return X_split_train, y_split_train, X_split_val, y_split_val
```

Step-by-Step:

1. Shuffle data randomly (prevents bias if data is ordered)
2. Calculate split point (typically 60-80% for training)
3. Use slicing to split into train and validation sets
4. Return both sets

Why Split:

- Train on training set
- Evaluate on validation/test set (unseen data)
- Prevents overfitting
- Gives realistic performance estimate

Plain English: Randomly shuffle data, then split into training (larger portion) and test (smaller portion) sets. This lets you train on one set and evaluate on completely independent data.

7 Machine Learning Specific Operations

7.1 Vectorization for Predictions

Loop vs Vectorized Prediction

BAD - Using Loop (Slow):

```
# For each example, compute prediction
predictions = np.zeros(m)
for i in range(m):
    predictions[i] = w[0]*X[i,0] + w[1]*X[i,1] + b
```

GOOD - Vectorized (Fast):

```
# Compute all predictions at once
predictions = np.dot(X, w) + b
```

Why Vectorization:

- 10-100x faster
- Uses optimized low-level code (C/Fortran)
- More readable
- Industry standard

Course Pattern:

```
# Compute model output for all examples
f_wb = np.dot(X, w) + b

# With sigmoid (logistic regression)
z = np.dot(X, w) + b
f_wb = sigmoid(z)  # sigmoid applied element-wise
```

Plain English: Never loop over training examples when computing predictions! Use matrix operations instead. `np.dot(X, w)` computes predictions for ALL examples in one shot.

7.2 Gradient Computation

Vectorized Gradient Computation

Mathematical Formula:

$$\frac{\partial J}{\partial w} = \frac{1}{m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$

$$\frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})$$

Course Implementation:

```python
def compute_gradient(X, y, w, b, lambda_=None):
    """
    Computes gradient for logistic regression

    Args:
      X : (ndarray (m,n)) - m examples, n features
      y : (ndarray (m,)) - labels
      w : (ndarray (n,)) - weights
      b : (scalar) - bias

    Returns:
      dj_db : (scalar) - gradient for b
      dj_dw : (ndarray (n,)) - gradient for w
    """
    m, n = X.shape                              # Get dimensions

    # Initialize gradients
    dj_dw = np.zeros(w.shape)                   # Init w gradient
    dj_db = 0.                                  # Init b gradient

    # 1. Compute predictions
    f_w = sigmoid(np.dot(X, w) + b)             # Predict ALL examples

    # 2. Compute errors
    err = f_w - y                               # Error for ALL examples

    # 3. Compute gradients
    dj_dw = (1/m) * np.dot(X.T, err)            # w gradient:  X transpose *
        errors
    dj_db = (1/m) * np.sum(err)                 # b gradient:  mean of errors

    return dj_db, dj_dw                         # Return both gradients
```

Key Vectorization:

```python
# X.T shape: (n, m)
# err shape: (m,)
# np.dot(X.T, err) shape: (n,)

# Breaking down np.dot(X.T, err):
# For feature j:
#   dj_dw[j] = sum of (err[i] * X[i,j]) for all i
# This is exactly the mathematical formula!
```

Plain English: Compute all predictions, get all errors, then one matrix multiplication gives all gradients! The magic is `np.dot(X.T, err)` which computes the sum in the gradient formula for all features at once.

7.3 Cost Functions

Linear Regression Cost (MSE)

Formula: Mean Squared Error

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})^2$$

Course Implementation:

```python
def compute_cost_mse(X, y, w, b):
    m = X.shape[0]                          # Get number of examples

    # Predictions
    f_wb = np.dot(X, w) + b                 # Compute ALL predictions

    # Errors
    err = f_wb - y                          # prediction - actual for ALL

    # Squared errors
    cost = np.sum(err ** 2)                 # Sum of squared errors

    # Average (with 1/2 factor)
    cost = cost / (2 * m)                   # Average over m examples

    return cost                             # Return single cost value
```

Plain English: Average of squared prediction errors. The `1/2` makes derivative cleaner (cancels with the 2 from squaring).

Logistic Regression Cost (Cross-Entropy)

Formula: Binary Cross-Entropy

$$J(w,b) = -\frac{1}{m}\sum_{i=1}^{m}[y^{(i)}\log(f_{w,b}(x^{(i)})) + (1-y^{(i)})\log(1-f_{w,b}(x^{(i)}))]$$

Course Implementation:

```python
def compute_cost(X, y, w, b, lambda_=1):
    m, n = X.shape                                          # Get dimensions

    # Clip for numerical stability
    logit = np.clip(np.dot(X, w) + b, -10, 10)        # Prevent overflow

    # Sigmoid predictions
    f_w = sigmoid(logit)                                    # Apply sigmoid

    # Cross-entropy (vectorized!)
    total_cost = -np.mean(y * np.log(f_w) + (1-y) * np.log(1-f_w))  # Cross-entropy loss

    return total_cost                                       # Return cost
```

Vectorization Breakdown:

```python
# y shape: (m,)
# f_w shape: (m,)

# Element-wise operations:
y * np.log(f_w)           # (m,) - cost when y=1
(1-y) * np.log(1-f_w)     # (m,) - cost when y=0

# Add them (element-wise)
# Then np.mean averages over all examples
```

Plain English: For logistic regression, use cross-entropy cost. When label is 1, cost is `-log(prediction)`. When label is 0, cost is `-log(1-prediction)`. This heavily penalizes confident wrong predictions!

7.4 Regularization

L2 Regularization

Purpose: Prevent overfitting by penalizing large weights
Regularized Cost:

```python
def compute_cost_reg(X, y, w, b, lambda_=1):
    # Compute base cost (without regularization)
    cost_without_reg = compute_cost(X, y, w, b)

    # Add regularization term: (lambda/2m) * sum(w^2)
    m = X.shape[0]
    reg_cost = (lambda_ / (2*m)) * np.sum(np.square(w))

    total_cost = cost_without_reg + reg_cost
    return total_cost
```

Regularized Gradient:

```python
def compute_gradient_reg(X, y, w, b, lambda_=1):
    m, n = X.shape

    # Compute base gradient
    dj_db, dj_dw = compute_gradient(X, y, w, b)

    # Add regularization to weight gradient (NOT bias!)
    dj_dw += (lambda_ / m) * w

    return dj_db, dj_dw
```

Key Points:

- Regularization term: $\frac{\lambda}{2m} \sum_j w_j^2$
- Adds to gradient: $\frac{\lambda}{m} w_j$
- Only regularize w, NOT b!
- Larger λ = more regularization (simpler model)

Plain English: Regularization adds a penalty for large weights. This encourages smaller, more distributed weights, preventing overfitting. The λ parameter controls strength: larger = more penalty.

8 Deep Copy

copy.deepcopy() - Avoid Modifying Original

The Problem:

```
import copy

w_original = np.array([1.0, 2.0, 3.0])

# BAD - Creates reference (shallow copy)
w = w_original
w = w - 0.1  # This MODIFIES w_original too!

# GOOD - Creates independent copy
w = copy.deepcopy(w_original)
w = w - 0.1  # w_original unchanged
```

Course Usage:

```
def gradient_descent(X, y, w_in, b_in, alpha, num_iters):
    # Avoid modifying the input parameters!
    w = copy.deepcopy(w_in)
    b = b_in

    for i in range(num_iters):
        dj_db, dj_dw = compute_gradient(X, y, w, b)
        w = w - alpha * dj_dw
        b = b - alpha * dj_db

    return w, b
```

Why Needed:

- Python variables are references, not copies
- `w = w_in` doesn't copy, just creates another name
- Modifying `w` would modify `w_in` too!
- `copy.deepcopy()` creates true independent copy

Plain English: Use `copy.deepcopy()` when you want to modify something without changing the original. Essential in gradient descent to avoid modifying input parameters!

9 Visualization with Matplotlib

9.1 Basic Plotting

plt.plot() - Line/Scatter Plot

Import:

```
import matplotlib.pyplot as plt
```

Basic Line Plot:

```
x = np.arange(-10, 10, 0.1)
y = sigmoid(x)

plt.plot(x, y, label="Sigmoid function")
plt.legend()
plt.show()
```

Scatter Plot with Different Markers:

```
# Separate classes
positive = (y_train == 1)
negative = (y_train == 0)

# Plot each class differently
plt.plot(X[positive, 0], X[positive, 1], 'k+', label="Admitted")
plt.plot(X[negative, 0], X[negative, 1], 'yo', label="Not admitted")

plt.xlabel('Exam 1 score')
plt.ylabel('Exam 2 score')
plt.legend(loc="upper right")
plt.show()
```

Marker Styles:

- `'k+'` - black plus signs
- `'yo'` - yellow circles
- `'r.'` - red dots
- `'b-'` - blue line
- `'g--'` - green dashed line

Plain English: `plt.plot(x, y)` plots y vs x. Add marker/color as third argument. Use `label` for legend, `xlabel/ylabel` for axis labels, `show()` to display.

Multiple Plots - Training Progress

Course Pattern:

```
# Plot training history
it = np.arange(0, num_iters, 1)

plt.plot(it, J_history_train, label="Training Cost")
plt.plot(it, J_history_train_acc, label="Training Accuracy")
plt.plot(it, J_history_test, label="Test Accuracy")

plt.xlabel('Iterations')
plt.ylabel('Value')
plt.legend()
plt.show()
```

What This Shows:

- Training cost should decrease over time
- Training accuracy should increase
- Test accuracy shows generalization
- Gap between train/test indicates overfitting

Plain English: Multiple `plt.plot()` calls on same axes create multiple lines. Use different labels to distinguish them in legend. Essential for monitoring training progress!

10 Common Course Patterns

10.1 Complete Training Loop

Full Gradient Descent Implementation

```python
def gradient_descent_val(X, y, w_in, b_in, cost_function,
                         gradient_function, alpha, num_iters, lambda_):
    """
    Complete gradient descent with train/val split
    """
    # 1. Shuffle and split data
    np.random.seed(123)
    perm = np.random.permutation(X.shape[0])
    X = X[perm, :]
    y = y[perm]

    size_train = int(0.66 * X.shape[0])
    X_train = X[0:size_train, :]
    y_train = y[0:size_train]
    X_val = X[size_train:, :]
    y_val = y[size_train:]

    # 2. Initialize
    J_history_train = []
    J_history_val = []
    w = copy.deepcopy(w_in)
    b = b_in

    # 3. Training loop
    for i in range(num_iters):
        # Compute gradient
        dj_db, dj_dw = gradient_function(X_train, y_train, w, b,
            lambda_)

        # Update parameters
        w = w - alpha * dj_dw
        b = b - alpha * dj_db

        # Track progress
        if i < 100000:
            J_history_train.append(cost_function(X_train, y_train, w, b
                ))
            J_history_val.append(accuracy(X_val, y_val, w, b))

        # Print progress
        if i % math.ceil(num_iters / 50) == 0:
            print(f"Iteration {i:4d}: Cost {J_history_train[-1]:8.2f}")

    # 4. Plot results
    it = np.arange(0, len(J_history_train), 1)
    plt.plot(it, J_history_train, label="Training Cost")
    plt.plot(it, J_history_val, label="Validation Accuracy")
    plt.legend()
    plt.show()

    return w, b, J_history_train, J_history_val
```

Key Steps:

1. Shuffle and split data (train/validation)
2. Initialize parameters and history tracking
3. Loop: compute gradient, update parameters, track progress

10.2 Prediction and Accuracy

Making Predictions

Logistic Regression Prediction:

```python
def predict(X, w, b):
    """
    Predict 0 or 1 using learned logistic regression parameters

    Args:
      X : (ndarray (m, n))
      w : (ndarray (n,)) - weights
      b : (scalar) - bias

    Returns:
      p : (ndarray (m,)) - predictions (0 or 1)
    """
    # Compute probabilities
    f_w = sigmoid(np.dot(X, w) + b)

    # Threshold at 0.5
    p = (f_w >= 0.5)  # Boolean array

    return p
```

Compute Accuracy:

```python
def accuracy(X, y, w, b):
    """
    Compute accuracy as percentage
    """
    p = (sigmoid(np.dot(X, w) + b) >= 0.5)
    return np.mean(p == y) * 100
```

Usage:

```python
# After training
predictions = predict(X_test, w, b)
acc = np.mean(predictions == y_test) * 100
print(f'Test Accuracy: {acc:.2f}%')
```

11 Debugging and Best Practices

Common Mistakes and How to Fix

1. Shape Mismatches

```
# Always check shapes!
print(f"X shape: {X.shape}")
print(f"w shape: {w.shape}")
print(f"y shape: {y.shape}")

# Common issue: (n,) vs (n,1)
w.shape  # (5,) - 1D array
w = w.reshape(-1, 1)  # Now (5, 1) - column vector
```

2. Forgetting to Normalize

```
# Always normalize features!
X = zscore_normalize_features(X)

# Or at minimum, check scales
print(f"Feature means: {np.mean(X, axis=0)}")
print(f"Feature stds: {np.std(X, axis=0)}")
```

3. Learning Rate Issues

```
# Plot cost over iterations
plt.plot(J_history)
plt.xlabel('Iterations')
plt.ylabel('Cost')
plt.show()

# If cost increases: alpha too large, reduce it
# If cost decreases slowly: alpha too small, increase it
```

4. Not Using Vectorization

```
# BAD - Loop (slow)
for i in range(m):
    predictions[i] = np.dot(w, X[i]) + b

# GOOD - Vectorized (fast)
predictions = np.dot(X, w) + b
```

5. Numerical Instability

```
# Clip values before exp/log
logit = np.clip(np.dot(X, w) + b, -10, 10)
f_w = sigmoid(logit)

# Avoid log(0)
cost = -np.mean(y * np.log(f_w + 1e-15) + (1-y) * np.log(1-f_w + 1e-15)
    )
```

12 Quick Reference Tables

12.1 NumPy Functions Summary

Function	Purpose	Example
`np.array()`	Create array	`np.array([1,2,3])`
`np.zeros()`	Array of zeros	`np.zeros((3,4))`
`np.arange()`	Sequence	`np.arange(0,10,0.1)`
`np.linspace()`	Evenly spaced	`np.linspace(0,1,100)`
`np.random.rand()`	Uniform [0,1)	`np.random.rand(5)`
`np.random.normal()`	Gaussian noise	`np.random.normal(0,1,100)`
`np.random.seed()`	Reproducibility	`np.random.seed(42)`
`np.random.permutation()`	Shuffle indices	`np.random.permutation(n)`
`np.dot()`	Matrix multiply	`np.dot(X, w)`
`np.sum()`	Sum elements	`np.sum(arr, axis=0)`
`np.mean()`	Average	`np.mean(arr, axis=0)`
`np.std()`	Std deviation	`np.std(arr, axis=0)`
`np.exp()`	Exponential	`np.exp(-z)`
`np.log()`	Natural log	`np.log(p)`
`np.square()`	Square elements	`np.square(w)`
`np.clip()`	Limit range	`np.clip(z, -10, 10)`
`np.loadtxt()`	Load from file	`np.loadtxt(url, delimiter=',')`
`.shape`	Get dimensions	`X.shape`
`.reshape()`	Change shape	`arr.reshape(-1,1)`
`.T`	Transpose	`X.T`

12.2 Common Shapes

Variable	Shape	Meaning
`X`	`(m, n)`	m examples, n features
`y`	`(m,)`	m labels (1D)
`w`	`(n,)`	n weights (1D)
`b`	scalar	bias (single number)
`X.T`	`(n, m)`	Transposed features
`np.dot(X, w)`	`(m,)`	m predictions
`np.dot(X.T, err)`	`(n,)`	gradient for w

12.3 Axis Parameter

Operation	Result
`np.sum(X)`	Sum all elements (scalar)
`np.sum(X, axis=0)`	Sum down columns (shape: `(n,)`)
`np.sum(X, axis=1)`	Sum across rows (shape: `(m,)`)
`np.mean(X, axis=0)`	Mean of each column (shape: `(n,)`)
`np.std(X, axis=0)`	Std dev of each column (shape: `(n,)`)

13 Appendix: Broadcasting Rules

NumPy Broadcasting

What is Broadcasting?
NumPy automatically "stretches" arrays to make operations work.
Example 1: Scalar + Array:

```
arr = np.array([1, 2, 3, 4])  # shape: (4,)
result = arr + 10              # shape: (4,)
# 10 is broadcast to [10, 10, 10, 10]
# Result: [11, 12, 13, 14]
```

Example 2: Array - Mean:

```
X = np.array([[1, 2],
              [3, 4],
              [5, 6]])         # shape: (3, 2)

X_mean = np.mean(X, axis=0)    # shape: (2,) = [3., 4.]
X_centered = X - X_mean        # Broadcasting!

# X_mean broadcast to [[3., 4.],
#                      [3., 4.],
#                      [3., 4.]]
# Result: [[-2., -2.],
#          [ 0.,  0.],
#          [ 2.,  2.]]
```

Rules:

1. If shapes differ in number of dimensions, pad with 1s on left
2. Arrays are compatible if dimensions are equal or one is 1
3. Where dimension is 1, stretch to match other array

Plain English: Broadcasting lets you do math between arrays of different shapes without explicitly copying data. NumPy figures out how to "stretch" smaller array to match larger one.

14 Summary

Essential Takeaways

Core NumPy Skills:

1. **Arrays**: Create with `np.array()`, `zeros()`, `arange()`, `linspace()`
2. **Indexing**: `arr[i]`, `arr[i:j]`, boolean masks
3. **Operations**: Element-wise `+,-,*,/`, matrix `np.dot()`
4. **Aggregation**: `sum()`, `mean()`, `std()` with `axis` parameter
5. **Math**: `exp()`, `log()`, `square()`, `clip()`
6. **Random**: `seed()`, `rand()`, `normal()`, `permutation()`
7. **Shapes**: Always check `.shape`, use `.reshape()` when needed

Machine Learning Workflow:

1. Load data: `np.loadtxt()`
2. Normalize: `zscore_normalize_features()`
3. Split: train/test using `permutation()` and slicing
4. Vectorize: Use `np.dot(X, w)` not loops!
5. Train: Gradient descent with `copy.deepcopy()`
6. Evaluate: Predictions and accuracy
7. Visualize: Plot training curves

Key Patterns to Remember:

- Predictions: `np.dot(X, w) + b`
- Gradient for w: `(1/m) * np.dot(X.T, err)`
- Gradient for b: `(1/m) * np.sum(err)`
- Accuracy: `np.mean(predictions == y) * 100`
- Always normalize features before training!

Golden Rules:

1. **Never loop over training examples** - vectorize!
2. **Always check shapes** - print them liberally
3. **Normalize features** - crucial for gradient descent
4. **Plot training curves** - catch problems early
5. **Use seed for reproducibility** - compare results